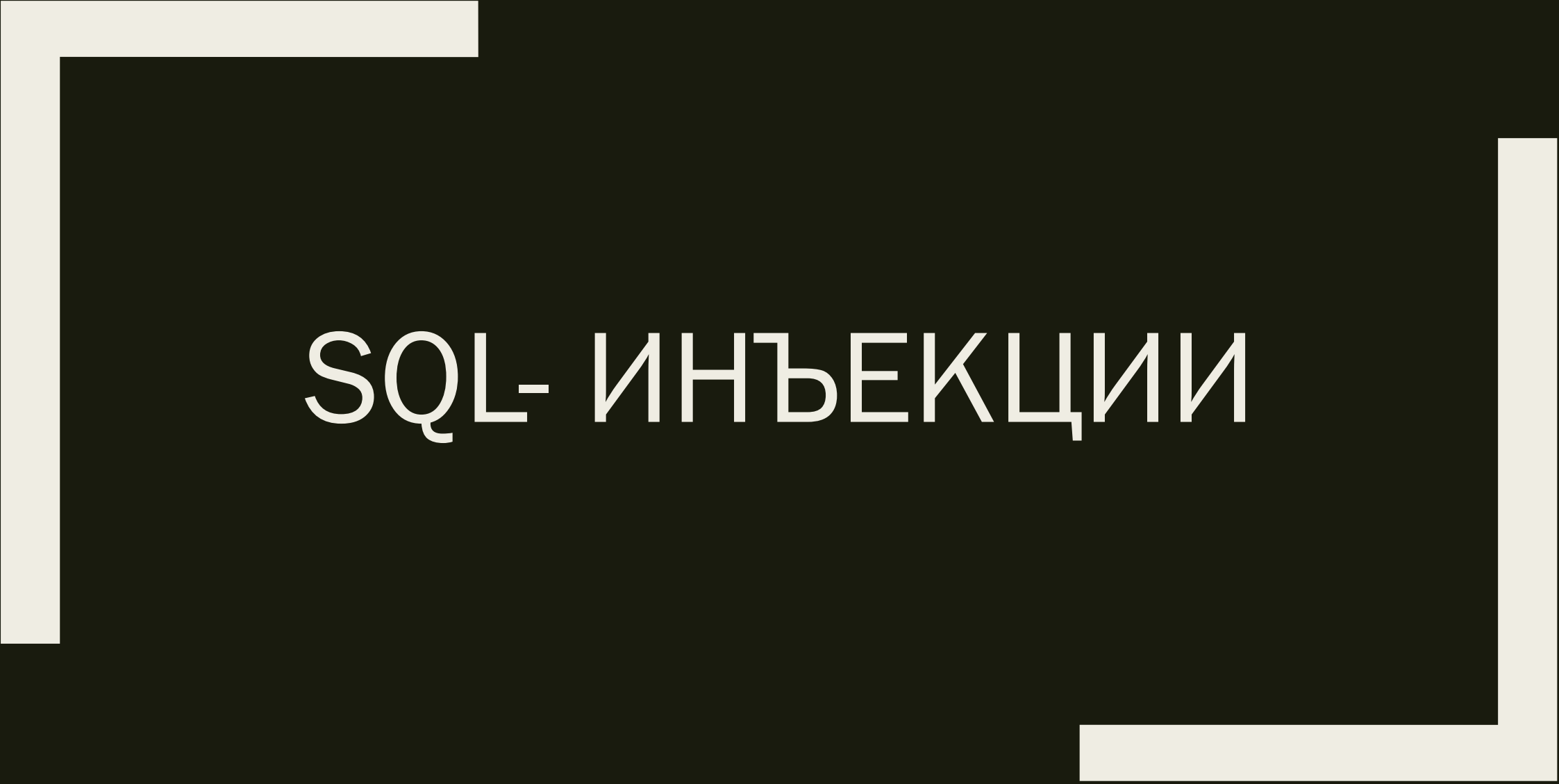


A decorative frame consisting of two thick, light-colored L-shaped lines. One line starts at the top-left and extends horizontally to the right, then vertically down. The other line starts at the bottom-right and extends horizontally to the left, then vertically up. They meet at the center, framing the text.

SQL-ИНЪЕКЦИИ

Внедрение SQL-кода

Внедрение SQL-кода (англ. SQL injection) — один из распространённых способов взлома сайтов и программ, работающих с базами данных, основанный на внедрении в запрос произвольного SQL-кода.

Внедрение SQL, в зависимости от типа используемой СУБД и условий внедрения, может дать возможность атакующему выполнить произвольный запрос к базе данных (например, прочитать содержимое любых таблиц, удалить, изменить или добавить данные), получить возможность чтения и/или записи локальных файлов и выполнения произвольных команд на атакуемом сервере.

Внедрение SQL-кода

Атака типа внедрения SQL может быть возможна из-за некорректной обработки входных данных, используемых в SQL-запросах.

Разработчик прикладных программ, работающих с базами данных, должен знать о таких уязвимостях и принимать меры противодействия внедрению SQL.

Принцип атаки внедрения SQL

Допустим, серверное ПО, получив входной параметр `id`, использует его для создания SQL-запроса. Рассмотрим следующий PHP-скрипт:

```
$id = $_REQUEST['id'];
```

```
$res = mysqli_query("SELECT * FROM news WHERE id_news = " . $id);
```

Принцип атаки внедрения SQL

Если на сервер передан параметр `id`, равный 5 (например так: `http://example.org/script.php?id=5`), то выполнится следующий SQL-запрос:

```
SELECT * FROM news WHERE id_news = 5
```

Принцип атаки внедрения SQL

Но если злоумышленник передаст в качестве параметра id строку -1 OR 1=1 (например, так: `http://example.org/script.php?id=-1+OR+1=1`), то выполнится запрос:

```
SELECT * FROM news WHERE id_news = -1 OR 1=1
```

Принцип атаки внедрения SQL

Таким образом, изменение входных параметров путём добавления в них конструкций языка SQL вызывает изменение в логике выполнения SQL-запроса (в данном примере вместо новости с заданным идентификатором будут выбраны все имеющиеся в базе новости, поскольку выражение $1=1$ всегда истинно — вычисления происходят по кратчайшему контуру в схеме).

Внедрение в строковые параметры

Предположим, серверное ПО, получив запрос на поиск данных в новостях параметром `search_text`, использует его в следующем SQL-запросе (здесь параметры экранируются кавычками):

```
$search_text = $_REQUEST['search_text'];  
$res = mysqli_query("SELECT id_news, news_date, news_caption, news_text,  
news_id_author FROM news WHERE news_caption LIKE('%$search_text%')");
```


Внедрение в строковые параметры

Сделав запрос вида `http://example.org/script.php?search_text=Test` мы получим выполнение следующего SQL-запроса:

```
SELECT id_news, news_date, news_caption, news_text, news_id_author FROM  
news WHERE news_caption LIKE('%Test%')
```

Внедрение в строковые параметры

Но, внедрив в параметр `search_text` символ кавычки (который используется в запросе), мы можем кардинально изменить поведение SQL-запроса. Например, передав в качестве параметра `search_text` значение `')+and+(news_id_author='1`, мы вызовем к выполнению запрос:

```
SELECT id_news, news_date, news_caption, news_text, news_id_author FROM  
news WHERE news_caption LIKE('%') and (news_id_author='1%')
```

Использование UNION

Язык SQL позволяет объединять результаты нескольких запросов при помощи оператора UNION. Это предоставляет злоумышленнику возможность получить несанкционированный доступ к данным.

Рассмотрим скрипт отображения новости (идентификатор новости, которую необходимо отобразить, передается в параметре id):

```
$res = mysqli_query("SELECT id_news, header, body, author FROM news WHERE  
id_news = " . $_REQUEST['id']);
```

Использование UNION

Если злоумышленник передаст в качестве параметра id конструкцию -1 UNION
SELECT 1,username, password,1 FROM admin, это вызовет выполнение SQL-запроса

```
SELECT id_news, header, body, author FROM news WHERE id_news = -1 UNION  
SELECT 1,username,password,1 FROM admin
```

Использование UNION

Так как новости с идентификатором `-1` заведомо не существует, из таблицы `news` не будет выбрано ни одной записи, однако в результат попадут записи, несанкционированно отображенные из таблицы `admin` в результате инъекции SQL.

Использование UNION + group_concat()

В некоторых случаях хакер может провести атаку, но не может видеть более одной колонки. В случае MySQL взломщик может воспользоваться функцией:

`group_concat(col, symbol, col)`

которая объединяет несколько колонок в одну. Например, для примера, данного выше вызов функции будет таким:

```
-1 UNION SELECT group_concat(username, 0x3a, password) FROM admin
```

Экранирование хвоста запроса

Зачастую SQL-запрос, подверженный данной уязвимости, имеет структуру, усложняющую или препятствующую использованию union. Например, скрипт

```
$res = mysqli_query("SELECT author FROM news WHERE id=" . $_REQUEST['id'] . "  
AND author LIKE ('a%')");
```

отображает имя автора новости по передаваемому идентификатору id только при условии, что имя начинается с буквы, а, и внедрение кода с использованием оператора UNION затруднительно.

Экранирование хвоста запроса

В таких случаях злоумышленниками используется метод экранирования части запроса при помощи символов комментария (`/*` или `--` в зависимости от типа СУБД).

В данном примере злоумышленник может передать в скрипт параметр `id` со значением `-1 UNION SELECT password FROM admin/*`, выполнив таким образом запрос

```
SELECT author FROM news WHERE id=-1 UNION SELECT password FROM admin/* AND  
author LIKE ('a%')
```

в котором часть запроса (`AND author LIKE ('a%')`) помечена как комментарий и не влияет на выполнение.

Расщепление SQL-запроса

Для разделения команд в языке SQL используется символ; (точка с запятой), внедряя этот символ в запрос, злоумышленник получает возможность выполнить несколько команд в одном запросе, однако не все диалекты SQL поддерживают такую возможность.

Например, если в параметры скрипта

```
$id = $_REQUEST['id'];
```

```
$res = mysqli_query("SELECT * FROM news WHERE id_news = $id");
```

Расщепление SQL-запроса

злоумышленником передается конструкция, содержащая точку с запятой, например `12;INSERT INTO admin (username, password) VALUES ('HaCkEr', 'foo');` то в одном запросе будут выполнены 2 команды

```
SELECT * FROM news WHERE id_news = 12;
```

```
INSERT INTO admin (username, password) VALUES ('HaCkEr', 'foo');
```

и в таблицу `admin` будет несанкционированно добавлена запись `HaCkEr`.

Методика атак типа внедрение SQL-кода

Поиск скриптов, уязвимых для атаки

На данном этапе злоумышленник изучает поведение скриптов сервера при манипуляции входными параметрами с целью обнаружения их аномального поведения.

Манипуляция происходит всеми возможными параметрами:

- ☐ Данными, передаваемыми через методы POST и GET
- ☐ Значениями [HTTP-Cookie]
- ☐ AUTH_USER и AUTH_PASSWORD (при использовании аутентификации)

Как правило, манипуляция сводится к подстановке в параметры символа одинарной (реже двойной или обратной) кавычки.

Методика атак типа внедрение SQL-кода

Аномальным поведением считается любое поведение, при котором страницы, получаемые до и после подстановки кавычек, различаются (и при этом не выведена страница о неверном формате параметров).

Наиболее частые примеры аномального поведения:

- ☐ выводится сообщение о различных ошибках;
- ☐ при запросе данных (например, новости или списка продукции) запрашиваемые данные не выводятся вообще, хотя страница отображается

и т. д. Следует учитывать, что известны случаи, когда сообщения об ошибках, в силу специфики разметки страницы, не видны в браузере, хотя и присутствуют в её HTML-коде.

Защита от атак типа внедрение SQL-кода

Фильтрация строковых параметров

Чтобы внедрение кода (закрытие строки, начинающейся с кавычки, другой кавычкой до её завершения текущей закрывающей кавычкой для разделения запроса на две части) было невозможно, для некоторых СУБД, в том числе, для MySQL, требуется брать в кавычки все строковые параметры. В самом параметре заменяют кавычки на \", апостроф - на \', обратную косую черту - на \\ (это называется «экранировать спецсимволы»).

Защита от атак типа внедрение SQL-кода

Усечение входных параметров

Для внесения изменений в логику выполнения SQL-запроса требуется внедрение достаточно длинных строк. Так, минимальная длина внедряемой строки в вышеприведённых примерах составляет 8 символов («1 OR 1=1»). Если максимальная длина корректного значения параметра невелика, то одним из методов защиты может быть максимальное усечение значений входных параметров.

Например, если известно, что поле id в вышеприведённых примерах может принимать значения не более 9999, можно «отрезать лишние» символы, оставив не более четырёх:

Защита от атак типа внедрение SQL-кода

Использование параметризованных запросов

Многие серверы баз данных поддерживают возможность отправки параметризованных запросов (подготовленные выражения). При этом параметры внешнего происхождения отправляются на сервер отдельно от самого запроса либо автоматически экранируются клиентской библиотекой.